



《AI大模型Ragent项目》——意图分数出来了，该查哪个库、查多少条

来自： 拿个offer-开源&项目实战

 马丁

2026年05月14日 21:54

开篇引言

上一篇讲了歧义引导——用户问退货政策，3C、家电和服装三个品类都举手了，系统停下来让用户选。handleGuidance 检测到歧义后短路输出引导文案，Pipeline 直接 return，不走检索。

歧义引导走完了——要么用户被问了一次做出了选择，要么意图本身明确不需要引导。不管哪种情况，到这一步意图分类的结果已经确定了。每个子问题手里拿着的意图列表是板上钉钉的：可能是一个 KB 意图（清关流程，score=0.92），可能是一个 MCP 意图（订单查询，score=0.90），也可能是一个 SYSTEM 意图（欢迎与问候，score=0.88）。

现在摆在面前的问题是：**这些意图分数怎么翻译成实际的动作？**

不同类型的意图走完全不同的路径——KB 意图要去向量数数据库的某个 Collection 做向量检索，MCP 意图要调订单系统的 API 拿实时数据，SYSTEM 意图压根不用查任何东西直接回复就行。而且分数的高低还会影响检索策略——高置信度走定向检索就够了，低置信度还得兜底搜全库给 Reranker 更多候选。

本篇是意图识别子系列的第 5 篇也是收尾篇。前四篇解决了怎么知道用户想问什么——树结构（第 5 篇）→ 打分（第 6 篇）→ 封顶（第 7 篇）→ 歧义引导（第 8 篇），本篇补上最后一环：**知道了之后怎么做。**

用电商客服的四个典型问题来跑一遍：

用户问题	命中意图	类型	接下来该做什么
你好	欢迎与问候 (0.88)	SYSTEM	直接回复，不走检索
帮我查一下订单 2024112801 的物流进度	订单查询 (0.90)	MCP	调工具，不走向量检索
跨境包裹清关一般要多久？	清关流程 (0.92)	KB	去对应 Collection 做定向检索
退货规则是什么	清关流程 (0.55)	KB	分数低于置信度阈值，定向 + 全局兜底检索

四个问题，四条路径，本篇逐个拆解。

总览：从意图到动作的分发地图

1. Pipeline 中的位置

先回到 StreamChatPipeline 的 execute() 方法，定位本篇涉及的阶段：

```
public void execute(StreamChatContext ctx) {
    loadMemory(ctx);           // 第 2 篇
    rewriteQuery(ctx);          // 第 4 篇
    resolveIntents(ctx);        // 第 5~7 篇

    if (handleGuidance(ctx)) {
        return;                // 第 8 篇：歧义引导短路
    }
    if (handleSystemOnly(ctx)) {
        return;                // ← 本篇重点：SYSTEM 意图短路
    }

    RetrievalContext retrievalCtx = retrieve(ctx); // ← 本篇重点：KB/MCP 分流入口
    if (handleEmptyRetrieval(ctx, retrievalCtx)) {
        return;
    }

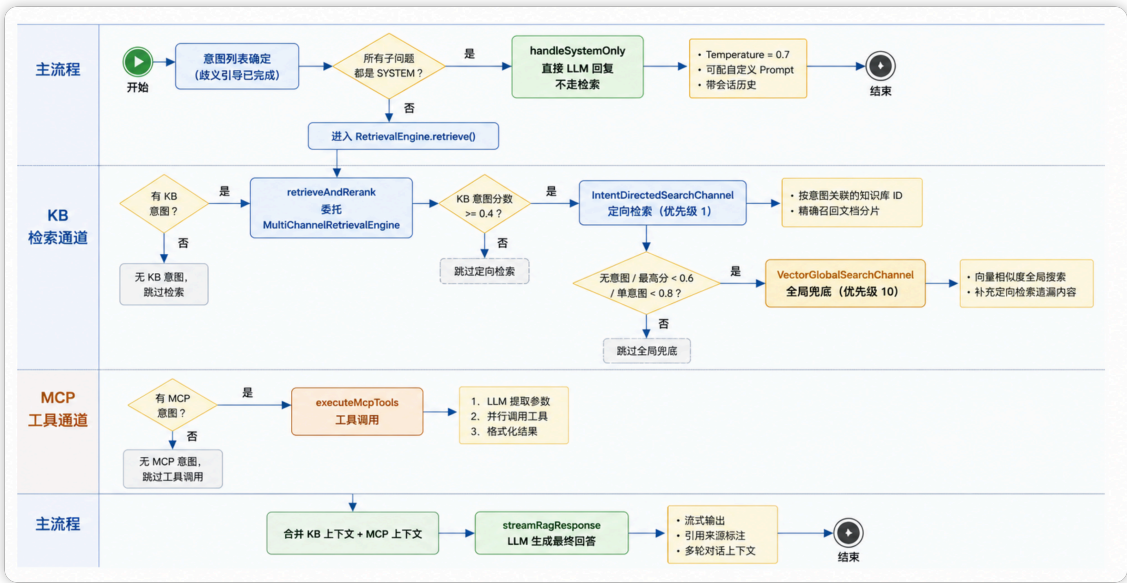
    streamRagResponse(ctx, retrievalCtx);
}
```

上一篇讲到 handleGuidance 返回 false（没有歧义或者引导完成了），Pipeline 继续往下走。接下来就是本篇的两个核心位置：

handleSystemOnly: SYSTEM 意图的短路分支，不走检索直接回复

retrieve: KB 意图和 MCP 意图的分流入口，委托给 RetrievalEngine 处理

2. 三条路径一张图



用一张表汇总三种意图类型的处理方式：

意图类型	处理方式	关键代码入口	是否走向量检索	备注
SYSTEM	短路直接回复	handleSystemOnly()	否	可配自定义 Prompt, Temperature=0.7
MCP	工具调用	executeMcpTools()	否	LLM 提取参数 + 工具执行
KB	定向 / 全局检索	retrieveAndRerank()	是	走多通道检索引擎, 可能两个通道同时激活

SYSTEM 意图：不查库，直接回复

1. 什么场景走这条路

用户发了一句“你好”。意图分类命中了 系统交互 > 欢迎与问候 节点 (kind=SYSTEM, score=0.88)。这种问题去向量库搜 chunk 纯属浪费——答案不在任何知识库文档里，而是系统自己该有的社交应答能力。搜出来的 chunk 不但没用，还可能干扰 LLM 的回复（比如搜到一段退货政策的文档片段，LLM 莫名其妙地在问好回复里塞进一句退货相关的话）。最合理的做法就是跳过检索，让 LLM 直接根据会话历史和系统提示词回复。

2. handleSystemOnly 的判定条件

```
private boolean handleSystemOnly(StreamChatContext ctx) {
    List<SubQuestionIntent> subIntents = ctx.getSubIntents();
    boolean allSystemOnly = subIntents.stream()
        .allMatch(si -> intentResolver.isSystemOnly(si.nodeScores()));
    if (!allSystemOnly) {
        return false;
    }
    // ... 短路处理
    return true;
}
```

判定的核心是 allMatch——必须所有子问题都是 SYSTEM-only 才短路。
isSystemOnly 的定义也很严格：

```
public boolean isSystemOnly(List<NodeScore> nodeScores) {
    return nodeScores.size() == 1
}
```

```
    && nodeScores.get(0).getNode() != null
    && nodeScores.get(0).getNode().getKind() == SYSTEM;
}
```

恰好只有一个意图，且 kind 是 SYSTEM，才算 SYSTEM-only。
为什么这么谨慎？看两个场景：

场景	子问题意图	是否短路	原因
用户说“你好”	[欢迎与问候 (SYSTEM, 0.88)]	✅ 短路	唯一意图是 SYSTEM
用户说“你好，顺便帮我查一下退货政策”	子问题 1: [欢迎与问候 (SYSTEM, 0.85)] 子问题 2: [退货政策 (KB, 0.78)]	❌ 不短路	子问题 2 不是 SYSTEM-only
用户说“谢谢”	[情感反馈 (SYSTEM, 0.82)]	✅ 短路	唯一意图是 SYSTEM

第二个场景很典型——用户打了个招呼顺带问了个问题，查询改写把它拆成两个子问题。虽然打招呼那个子问题是 SYSTEM，但退货政策那个子问题是 KB，allMatch 不满足，走正常流程。这样退货政策的问题不会被跳过。

3. 自定义 Prompt 和直接回复

判定通过后的处理逻辑：

```
String customPrompt = subIntents.stream()
    .flatMap(si -> si.nodeScores().stream())
    .map(ns -> ns.getNode().getPromptTemplate())
    .filter(StrUtil::isNotBlank)
    .findFirst()
    .orElse(null);

StreamCancellationHandle handle = streamSystemResponse(
    ctx.getRewriteResult().rewrittenQuestion(),
    ctx.getHistory(),
    customPrompt,
    ctx.getCallback()
);
```

两个设计点值得注意。
自定义 Prompt 优先。 SYSTEM 节点上可以配 promptTemplate。比如欢迎与问候节点配了一段用活泼亲切的语气和用户打招呼，介绍自己的能力范围——这比默认的通用 System Prompt 更贴合场景。findFirst 找到第一个非空的自定义模板就用，没配就退回到全局默认。
会话历史照常带入。 streamSystemResponse 的实现里把 history 完整传入了消息列表：

```
private StreamCancellationHandle streamSystemResponse(String question, List<ChatMessage> history,
                                                    String customPrompt, StreamCallback callback) {
    String systemPrompt = StrUtil.isNotBlank(customPrompt)
        ? customPrompt
        : promptTemplateLoader.load(CHAT_SYSTEM_PROMPT_PATH);

    List<ChatMessage> messages = new ArrayList<>();
    messages.add(ChatMessage.system(systemPrompt));
    if (CollUtil.isNotEmpty(history)) {
        messages.addAll(history);
    }
    messages.add(ChatMessage.user(question));

    ChatRequest req = ChatRequest.builder()
        .messages(messages)
        .temperature(0.7D)
        .thinking(false)
        .build();
    return llmService.streamChat(req, callback);
}
```

不走检索不代表没有上下文。用户连续说了三句“你好”“你是谁”“你能做什么”，每次都短路到 SYSTEM 分支，但会话历史带着前面的对话，LLM 能理解上下文，不会每次都从头自我介绍。

4. Temperature 的差异

注意 temperature=0.7D。和后面 KB 检索场景的 temperature=0D 形成了鲜明对比。

场景	Temperature	原因
SYSTEM 直接回复	0.7	闲聊需要自然度，“你好”不能每次回一模一样的话
KB 检索回复	0.0	必须严格忠实于检索到的 chunk，不能编造细节
MCP 工具回复	0.3	工具返回结构化数据，需要一点灵活性来组织自然语言回答

5. 短路的成本节省

对比 SYSTEM 短路和正常 RAG 流程的开销：

阶段	正常流程	SYSTEM 短路
会话记忆加载	✅ 已执行	✅ 已执行
查询改写	✅ 已执行	✅ 已执行
意图分类	✅ 已执行	✅ 已执行
歧义检测	✅ 已执行	✅ 已执行
向量检索 + BM25	✅ 已执行	❌ 跳过
Reranker 重排序	✅ 已执行	❌ 跳过
LLM 生成	✅ 已执行 (Prompt 长, ~2s)	✅ 已执行 (Prompt 短, ~0.5s)

跳过了检索和 Reranker 的 IO 开销，LLM 的 Prompt 也短了很多（不用塞 chunk），整体响应速度快了一大截。

handleGuidance（第 8 篇）和 handleSystemOnly（本篇）在 Pipeline 中是先后关系。先检查歧义引导，再检查 SYSTEM 短路，顺序是有讲究的。虽然 SYSTEM 意图不会参与歧义引导（歧义引导只看 KB 意图），但把歧义引导放前面是一个防御性设计——万一后续业务逻辑变化导致 SYSTEM 意图也产生歧义，这个顺序能保证歧义引导有机会先拦截。

MCP 意图：不查库，调工具

1. 什么场景走这条路

用户发了一句“帮我查一下订单 2024112801 的物流进度”。意图分类命中了 订单管理 > 订单查询 节点 (kind=MCP, mcpToolId=order-query-tool, score=0.90)。

物流进度是实时数据，存在订单系统里，不在知识库的向量数据库里。去向量数据库搜只能搜到如何查询物流进度这类帮助文档，搜不到这个具体订单的实际状态。正确的做法是调用订单查询工具，拿到实时数据后让 LLM 组织成自然语言回答。

2. KB 和 MCP 的分流

MCP 意图不走 handleSystemOnly 的短路分支 (因为 kind 不是 SYSTEM)，它和 KB 意图一起进入 retrieve() → RetrievalEngine.retrieve()。分流发生在更下层的 buildSubQuestionContext 方法里：

```
private SubQuestionContext buildSubQuestionContext(SubQuestionIntent intent, int topK) {
    List<NodeScore> kbIntents = NodeScoreFilters.kb(intent.nodeScores());
    List<NodeScore> mcpIntents = NodeScoreFilters.mcp(intent.nodeScores());

    KbResult kbResult = retrieveAndRerank(intent, kbIntents, topK);

    String mcpContext = CollUtil.isEmpty(mcpIntents)
        ? executeMcpAndMerge(intent.subQuestion(), mcpIntents)
        : "";

    return new SubQuestionContext(intent.subQuestion(), kbResult.groupedContext(), mcpContext, kbR
}
```

NodeScoreFilters 是分流的关键工具类，提供了三个静态方法：mcp() 过滤 MCP 意图，kb() 过滤 KB 意图，kb(scores, minScore) 在过滤 KB 意图的基础上再加一道分数门槛。后面讲 IntentDirectedSearchChannel 的时候会用到带 minScore 的重载版本——不是所有 KB 意图都值得走一次定向检索，分数太低的直接过滤掉。

```
public static List<NodeScore> mcp(List<NodeScore> scores) {
    return scores.stream()
        .filter(ns -> ns.getNode() != null && ns.getNode().isMcp())
        .filter(ns -> StrUtil.isNotBlank(ns.getNode().getMcpToolId()))
        .toList();
}

public static List<NodeScore> kb(List<NodeScore> scores) {
    return scores.stream()
        .filter(ns -> ns.getNode() != null && ns.getNode().isKB())
        .toList();
}

public static List<NodeScore> kb(List<NodeScore> scores, double minScore) {
    return scores.stream()
        .filter(ns -> ns.getScore() >= minScore)
        .filter(ns -> ns.getNode() != null && ns.getNode().isKB())
        .toList();
}
```

MCP 过滤有两个条件：kind==MCP 且 mcpToolId 非空。第二个条件是防御性检查。

注意 KB 和 MCP 是并行处理的，不是二选一。一个子问题的意图列表里可能同时有 KB 和 MCP 意图（第 7 篇的封顶算法保留了多个），两条路各走各的，最后合并。

3. 工具调用链路

MCP 意图进入 executeMcpAndMerge 后，走了一条和向量检索完全不同的链路。

3.1 从意图节点找到工具执行器

McpToolRegistry 是一个工具注册表，维护 toolId → McpToolExecutor 的映射。每个 McpToolExecutor 封装了一个 MCP 工具的完整调用逻辑——工具定义（参数 Schema、描述）和执行方法。

3.2 LLM 提取参数

找到工具后，下一步是从用户问题里提取参数：

```
String customParamPrompt = intentNode.getParamPromptTemplate();
Map<String, Object> params = mcpParameterExtractor.extractParameters(question, tool, customParamPr
```

McpParameterExtractor.extractParameters() 会调用一次 LLM，让模型根据工具的参数 Schema 和用户问题提取出参数。比如用户问“帮我查订单 2024112801 的物流进度”，LLM 提取出 {"orderId": "2024112801"}。

这里又调了一次 LLM——和意图分类的那次不同，这次是专门做参数提取的。意图节点上的 paramPromptTemplate 是可选的自定义提取提示词，可以针对特定工具定制提取规则。比如订单查询工具的节点配了一段“从用户问题中提取订单号，订单号通常是数字开头的 10~20 位字符串”。

3.3 执行工具

参数提取完，构建好入参然后调用工具：

```
private Map<String, List<McpSchema.CallToolResult>> executeMcpTools(String question,
                                                                    List<NodeScore> mcpIntentScores) {
    if (CollUtil.isEmpty(mcpIntentScores)) {
        return Map.of();
    }

    List<CompletableFuture<ToolOutput>> futures = mcpIntentScores.stream()
        .map(ns -> CompletableFuture.supplyAsync(
            () -> {
                String toolId = ns.getNode().getMcpToolId();
                try {
                    McpSchema.CallToolResult result = executeSingleMcpTool(question, ns.getToolSchema());
                    return result == null ? null : new ToolOutput(toolId, result);
                } catch (Exception e) {
                    log.error("MCP 工具调用异常, toolId: {}", toolId, e);
                    return new ToolOutput(toolId, McpSchema.CallToolResult.builder()
                        .content(List.of(new McpSchema.TextContent("工具调用异常: " + e.getMessage(), true))
                        .build());
                }
            },
            mcpBatchExecutor
        ))
        .toList();

    return futures.stream()
        .map(CompletableFuture::join)
        .filter(Objects::nonNull)
        .collect(Collectors.groupingBy(
            ToolOutput::toolId,
            Collectors.mapping(ToolOutput::result, Collectors.toList())
        ));
}
```

几个值得注意的设计：

并行调用：多个 MCP 意图通过 CompletableFuture + mcpBatchExecutor 并行调用，和 KB 检索用的是不同的线程池（mcpBatchExecutor vs ragContextExecutor），互不干扰。

单工具失败不阻断：try-catch 在每个 future 内部，一个工具调用失败返回失败的 ToolOutput，其他工具不受影响。

3.4 结果格式化

工具调用完成后，`contextFormatter.formatMcpContext()` 把 `CallToolResult` 格式化上下文片段，和 KB 检索的 chunk 一起塞进最终的 LLM Prompt。用户看到的是 LLM 基于工具返回的数据组织出来的自然语言回答——您的订单 2024112801 目前已发出，预计明天到达。

4. MCP 调用的延迟预期

MCP 工具调用涉及两次外部 IO：一次 LLM 参数提取（500ms），一次实际工具调用（取决于外部系统响应时间，通常 100500ms）。好消息是两个 MCP 意图的调用是并行的，总延迟取决于最慢的那个。

KB 意图：查知识库，两条通道

这是本篇最核心的章节——KB 意图的检索映射。

1. 从意图到检索通道

KB 意图的检索路径比 SYSTEM 和 MCP 都复杂，因为它涉及一个关键的决策：用定向检索还是全局检索，还是两个都用？

先看 `retrieveAndRerank` 的代码：

```
private KbResult retrieveAndRerank(SubQuestionIntent intent, List<NodeScore> kbIntents, int topK)
// 委托给多通道检索引擎
List<SubQuestionIntent> subIntents = List.of(intent);
List<RetrievedChunk> chunks = multiChannelRetrievalEngine.retrieveKnowledgeChannels(subIntents

if (CollUtil.isEmpty(chunks)) {
    return KbResult.empty();
}

// 按意图节点分组
Map<String, List<RetrievedChunk>> intentChunks = new HashMap<>();
if (CollUtil.isNotEmpty(kbIntents)) {
    for (NodeScore ns : kbIntents) {
        intentChunks.put(ns.getNode().getId(), chunks);
    }
} else {
    intentChunks.put(MULTI_CHANNEL_KEY, chunks);
}

String groupedContext = contextFormatter.formatKbContext(kbIntents, intentChunks, topK);
return new KbResult(groupedContext, intentChunks);
}
```

KB 意图的检索不是 `RetrievalEngine` 自己做的，而是委托给了 `MultiChannelRetrievalEngine`（多通道检索引擎）。这个引擎内部维护了两个检索通道（`SearchChannel`），它们根据意图的置信度动态决定是否激活：

`IntentDirectedSearchChannel`（意图定向检索，优先级 1）：根据意图节点的 `collectionName` 做精准检索，最快最准

`VectorGlobalSearchChannel`（向量全局检索，优先级 10）：在所有 `Collection` 中搜索，作为兜底安全网

两个通道不是二选一——根据条件，可能只激活定向检索，可能只激活全局检索，也可能两个同时激活。

2. 意图定向检索：IntentDirectedSearchChannel

2.1 激活条件

```
@Override
public boolean isEnabled(SearchContext context) {
    if (!properties.getChannels().getIntentDirected().isEnabled()) {
        return false;
    }
    if (CollUtil.isEmpty(context.getIntent())) {
        return false;
    }
    List<NodeScore> kbIntents = extractKbIntents(context);
```

```
return CollUtil.isEmpty(kbIntents);
}

private List<NodeScore> extractKbIntents(SearchContext context) {
    double minScore = properties.getChannels().getIntentDirected().getMinIntentScore();
    List<NodeScore> allScores = context.getIntent().stream()
        .flatMap(si -> si.nodeScores().stream())
        .toList();
    return NodeScoreFilters.kb(allScores, minScore);
}
```

三个条件：配置开关打开、有子问题意图、有 KB 意图且分数 $\geq$ minIntentScore（默认 0.4）。

注意这里的最低分数线是 0.4，比第 6 篇意图分类阶段的 INTENT_MIN_SCORE=0.35 高了一截。0.35 只是保留在意图列表里的底线——分数够低但不至于丢掉；0.4 才是值得走一次定向检索的门槛。一次定向检索意味着一次向量数据库 IO，不能太随便。

用电场景举两个例子：

清关流程意图 (score=0.92)：0.92 $\geq$ 0.4 ，定向检索激活

某个边缘意图 (score=0.38)：0.38 < 0.4 ，不走定向检索

2.2 检索过程

激活后，定向检索对每个 KB 意图执行：

```
@Override
public SearchChannelResult search(SearchContext context) {
    List<NodeScore> kbIntents = extractKbIntents(context);
    int topKMultiplier = properties.getChannels().getIntentDirected().getTopKMultiplier();
    List<RetrievedChunk> allChunks = retrieveByIntents(
        context.getMainQuestion(), kbIntents, context.getTopK(), topKMultiplier
    );
    // ...
}
```

retrieveByIntents 内部是 IntentParallelRetriever，它对每个 KB 意图做一件事：取 IntentNode.collectionName，在对应的 Milvus Collection 中做向量检索。

回忆第 5 篇讲过的 IntentNode 字段——每个 KB 类叶子节点绑定了一个 collectionName（如 kb_1997857139737882625），这个 Collection 就是这个意图对应的知识库在向量数据库中的存储位置。意图定向检索的精确性就来自这里：不是搜全库，而是只搜命中意图绑定的那个 Collection。

3. 全局兜底检索：VectorGlobalSearchChannel

3.1 三个启用条件

全局检索不是默认开启的。它只在意图置信度不够高的时候作为安全网介入。看 isEnabled 的三个条件：

```
@Override
public boolean isEnabled(SearchContext context) {
    if (!properties.getChannels().getVectorGlobal().isEnabled()) {
        return false;
    }

    List<NodeScore> allScores = context.getIntent().stream()
        .flatMap(si -> si.nodeScores().stream())
        .toList();

    // 条件1：没有识别出任何意图
    if (CollUtil.isEmpty(allScores)) {
        log.info("未识别出任何意图，启用全局检索");
        return true;
    }

    double maxScore = allScores.stream()
        .mapToDouble(NodeScore::getScore).max().orElse(0.0);
```



```
// 条件2: 最高分低于置信度阈值 (默认 0.6)
double threshold = properties.getChannels().getVectorGlobal().getConfidenceThreshold();
if (maxScore < threshold) {
    log.info("意图置信度过低 ({}), 启用全局检索", maxScore);
    return true;
}

// 条件3: 只有一个意图且分数低于补充阈值 (默认 0.8)
double supplementThreshold = properties.getChannels().getVectorGlobal()
    .getSingleIntentSupplementThreshold();
if (allScores.size() == 1 && maxScore < supplementThreshold) {
    log.info("单一中等置信度意图 ({}), 启用补充全局检索", maxScore);
    return true;
}

return false;
}
```

逐个条件拆解。

条件 1：没有识别出任何意图。

用户问“你们公司的知识产权归属条款”——意图树里没有匹配的节点，LLM 返回了空数组。定向检索无从下手，不知道该查哪个 Collection。全局检索是唯一的兜底——在所有知识库中搜一遍，也许能找到沾边的内容。

条件 2：最高分低于置信度阈值（默认 0.6）。

用户问“退货规则是什么”——最高分意图只有 0.55，系统不太确定用户到底想问哪个方向。0.55 低于 0.6 的置信度阈值，全局检索介入。这个场景有个关键细节：**定向检索也可能同时激活**。0.55 >= minIntentScore (0.4)，定向检索的激活条件也满足。两个通道同时跑——定向检索在命中意图的 Collection 里搜，全局检索在所有 Collection 里搜，结果合并后一起进入后置处理器（Reranker）。

条件 3：单意图 + 分数低于补充阈值（默认 0.8）。

再换一个场景。假设用户同样问“跨境包裹清关一般要多久”，但这次上下文不同（比如对话历史里有干扰信息），LLM 给出的置信度没那么高——清关流程意图只拿到 score=0.72（不像概览表里那个 0.92 的高分场景）。0.72 高于置信度阈值 0.6，条件 2 不触发；但低于补充阈值 0.8，条件 3 触发。系统觉得只靠一个中等置信度的意图不够稳，开启全局检索做补充。

为什么单意图场景要更加谨慎？因为只有一个意图意味着没有备选方案——如果这个意图判断错了，下游只能拿到错误 Collection 的 chunk，答案质量会很差。补充全局检索相当于买了一份保险。

用一张表汇总三个条件：

条件	触发场景	定向检索是否同时激活	电商场景举例
无意图	LLM 返回空数组	否（没有 KB 意图）	你们公司的知识产权归属条款
最高分 < 0.6	意图不太确定	可能（分数 >= 0.4 就走）	退货规则是什么（最高分 0.55）
单意图 < 0.8	中等置信度孤军	是（分数 >= 0.4）	跨境包裹清关一般要多久？（0.72）

3.2 全局检索做了什么

全局检索的行为比较直观——把所有知识库都搜一遍：

```
private List<String> getAllKBCollections() {
    List<KnowledgeBaseDO> kbList = knowledgeBaseMapper.selectList(
        Wrappers.lambdaQuery(KnowledgeBaseDO.class)
            .select(KnowledgeBaseDO::getCollectionName)
            .eq(KnowledgeBaseDO::getDeleted, 0)
    );
    // ... 收集所有 collectionName
}
```

从 KnowledgeBaseMapper 查出所有知识库的 collectionName，然后在所有 Collection 中并行做向量检索。全局检索的 topKMultiplier=3，比定向检索的 topKMultiplier=2 更大——全库搜索噪声更多，需要更大的候选池给 Reranker 精排。

全局检索内部怎么并行、结果怎么合并、Reranker 怎么精排，这些细节是第 10 篇的内容。本篇只关心什么条件下激活全局检索。

4. 两个通道的协同：一个完整的演算

用一个具体场景走一遍完整的决策过程。

用户问“退货规则是什么”，意图分类返回了清关流程（score=0.55）。

Step 1: 进入 MultiChannelRetrievalEngine

两个通道各自检查激活条件：

```
IntentDirectedSearchChannel.isEnabled()
    → minIntentScore=0.4, score=0.55 >= 0.4 → 启用 ✓

VectorGlobalSearchChannel.isEnabled()
    → 条件2: maxScore=0.55 < confidenceThreshold=0.6 → 启用 ✓
```

两个通道同时激活。

Step 2: 并行执行

```
List<SearchChannel> enabledChannels = searchChannels.stream()
    .filter(channel -> channel.isEnabled(context))
    .sorted(Comparator.comparingInt(SearchChannel::getPriority))
    .toList();

List<CompletableFuture<SearchChannelResult>> futures = enabledChannels.stream()
    .map(channel -> CompletableFuture.supplyAsync(
        () -> channel.search(context),
        ragRetrievalExecutor
    ))
    .toList();
```

两个通道通过 CompletableFuture 并行提交到 ragRetrievalExecutor 线程池：

定向检索：在清关流程绑定的 Collection 中检索，TopK = $10 \times 2 = 20$

全局检索：在所有 Collection 中检索，TopK = $10 \times 3 = 30$

Step 3: 结果合并 → 后置处理器链

两个通道的结果合并后进入后置处理器链——去重、Reranker 精排、TopK 截断，最终筛出最相关的几条 chunk。

这就是安全网设计的价值。定向检索赌对了（清关流程确实是正确方向），它的结果在 Reranker 中会得高分；定向检索赌错了（用户其实问的不是清关流程），全局检索从其他 Collection 补回正确的 chunk。两条路并行跑不白费，Reranker 做最后的裁判。

动态 TopK：节点配置 vs 全局默认

1. 两级 TopK 计算

TopK 的计算不是一个固定数字，而是分两级动态决定的。

第一级：子问题级。 RetrievalEngine 在处理每个子问题时，先看子问题的 KB 意图节点上有没有配 topK：

```
private int resolveSubQuestionTopK(SubQuestionIntent intent, int fallbackTopK) {
    return NodeScoreFilters.kb(intent.nodeScores()).stream()
        .map(NodeScore::getNode)
        .filter(Objects::nonNull)
        .map(IntentNode::getTopK)
        .filter(Objects::nonNull)
        .filter(topK -> topK > 0)
        .max(Integer::compareTo)
        .orElse(fallbackTopK);
}
```

逻辑很直白：从所有 KB 意图节点中取最大的 topK 配置值。如果没有任何节点配了 topK，用全局默认值 DEFAULT_TOP_K=10。

第二级：意图级。 IntentParallelRetriever 在处理每个具体意图时，把基础 TopK 乘以倍率：

```
private int resolveIntentTopK(NodeScore nodeScore, int fallbackTopK, int topKMultiplier) {
    int baseTopK = fallbackTopK;
    if (nodeScore != null && nodeScore.getNode() != null) {
```

```
Integer nodeTopK = nodeScore.getNode().getTopK();
if (nodeTopK != null && nodeTopK > 0) {
    baseTopK = nodeTopK;
}

if (topKMultiplier <= 0) {
    return baseTopK;
}

return baseTopK * topKMultiplier;
}
```

用一张表格说清楚两级计算的关系：

计算层级	方法	输入	输出	含义
第一级：子问题级	resolveSubQuestionTopK	子问题所有 KB 意图节点的 topK 配置	baseTopK	取节点中最大的 topK，无配置则用 DEFAULT_TOP_K=10
第二级：意图级	resolveIntentTopK	baseTopK + topKMultiplier	finalTopK	baseTopK × 倍率，得到实际检索数量

2. 用具体数字算一遍

场景 A——节点配了 topK：

清关流程节点配了 topK=5，定向检索的 topKMultiplier=2。

- 第一级：resolveSubQuestionTopK 取到清关流程的 topK=5，返回 5
- 第二级：resolveIntentTopK → baseTopK=5 × topKMultiplier=2 → finalTopK=10
- 实际执行：在清关流程的 Collection 中检索 Top-10

场景 B——节点没配 topK：

售后维修节点没配 topK（null）。

- 第一级：resolveSubQuestionTopK 没找到有效配置，退回 fallbackTopK=10
- 第二级：resolveIntentTopK → baseTopK=10 × topKMultiplier=2 → finalTopK=20
- 实际执行：在售后维修的 Collection 中检索 Top-20

3. 为什么节点可以配 topK

不同知识库的内容密度差别很大。退货政策知识库可能只有几十条 chunk，内容集中，5 条就能覆盖主要规则。跨境物流知识库涉及多个国家不同的政策法规，chunk 数量多且分散，需要 15 条以上才能覆盖到用户关心的点。统一用全局默认的 10 条，对退货政策来说太多（夹杂了冗余信息），对跨境物流来说又太少（漏掉了关键内容）。允许节点级配置，让每个知识库可以按自己的内容特点调整召回数量。

4. 为什么还要乘以倍率

定向检索 topKMultiplier=2，全局检索 topKMultiplier=3。
乘以倍率是因为检索阶段拿到的候选还要经过后置处理器链（第 10 篇会讲）——去重、Reranker 精排、TopK 截断。先多召回一些给 Reranker 选，最终塞进 LLM Prompt 的 chunk 数量由 Reranker 控制，不是这里的 TopK 直接决定的。
全局检索的倍率比定向检索大，是因为全局检索在所有 Collection 上搜，噪声更多，需要更大的候选池才能保证 Reranker 有足够的好候选可选。

LLM 生成阶段的参数差异

KB 和 MCP 的结果都拿到之后，合并成 RetrievalContext，传给 streamRagResponse 做最终的 LLM 生成。这里有一个容易忽略的细节——LLM 的参数会根据意图类型做差异化配置：

```
ChatRequest chatRequest = ChatRequest.builder()
    .messages(messages)
    .thinking(deepThinking)
    .temperature(ctx.hasMcp() ? 0.3D : 0D)    // MCP 场景稍微放宽温度
```

```
.topP(ctx.hasMcp() ? 0.8D : 1D)
.build();
```

纯 KB 场景（没有 MCP 意图），temperature=0。严格忠实于检索到的 chunk 内容，不允许 LLM 发挥创造力编造细节。这和第 6 篇意图分类的 temperature=0.1 类似，分析性任务需要确定性。

有 MCP 意图的场景，temperature=0.3。MCP 工具返回的是结构化数据（如 {"orderId": "2024112801", "status": "已发出", "estimatedArrival": "2026-04-29"}），LLM 需要一点灵活性把这些数据组织成自然语言——您的订单已发出，预计 4 月 29 日到达。太死板的话，LLM 可能只是把 JSON 原样输出。

配置参数一览

本篇涉及的配置参数比较多，整理一张汇总表方便查阅：

配置项	默认值	含义	调优方向
rag.search.channels.intentDirected.enabled	true	是否启用意图定向检索	关闭后所有 KB 意图只走全局检索
rag.search.channels.intentDirected.minIntentScore	0.4	定向检索的意图最低分数线	调高减少低置信度的误检索，调低放宽门槛
rag.search.channels.intentDirected.topKMultiplier	2	定向检索 TopK 倍率	调高增加 Reranker 候选池
rag.search.channels.vectorGlobal.enabled	true	是否启用全局兜底检索	关闭后低置信度场景可能检索不到内容
rag.search.channels.vectorGlobal.confidenceThreshold	0.6	全局检索的置信度触发阈值	调低减少兜底频率，调高增加安全网覆盖
rag.search.channels.vectorGlobal.singleIntentSupplementThreshold	0.8	单意图补充检索触发阈值	调低减少补充检索频率
rag.search.channels.vectorGlobal.topKMultiplier	3	全局检索 TopK 倍率	调高增加候选池但增加 Reranker 负担
rag.search.default_top_k	10	全局默认检索数量	作为未配置节点的 baseTopK

几个参数之间的关系容易混淆：INTENT_MIN_SCORE=0.35（第 6 篇，意图分类阶段的保留门槛）< minIntentScore=0.4（本篇，定向检索的激活门槛）< confidenceThreshold=0.6（本篇，全局兜底的触发阈值）< singleIntentSupplementThreshold=0.8（本篇，单意图补充的触发阈值）。四个阈值递增，对应的决策层级也递进——保留 → 检索 → 兜底 → 补充。

小结和预告

1. 意图识别子系列回顾

五篇下来，意图识别阶段的完整链路串起来了：

- 1. 第 5 篇——为什么要设计意图树。用三级树形结构（DOMAIN → CATEGORY → TOPIC）替代扁平四分类，解决多知识库场景下查哪个库的路由问题。
- 2. 第 6 篇——怎么给节点打分。把叶子节点序列化后用 Prompt 模板喂给 LLM，一次调用返回所有节点的 ID 和分数，五层容错保证 JSON 解析的鲁棒性。
- 3. 第 7 篇——多意图怎么封顶。capTotalIntents 三段式策略——多样性优先保证每个子问题至少一个代表，剩余配额按分数全局竞争。
- 4. 第 8 篇——歧义了怎么引导。三重过滤检测同名跨品类歧义，短路输出引导文案让用户选择，智能跳过避免重复打扰。
- 5. 第 9 篇（本篇）——分数出来了怎么做。SYSTEM 短路直接回复不走检索，MCP 触发工具调用拿实时数据，KB 走定向检索或全局兜底，低置信度场景两个通道同时跑 Reranker 做最后裁判。

从数据结构到打分规则，从封顶算法到歧义引导，最后翻译成实际的检索、工具调用和直接回复——意图识别阶段的故事到这里就讲完了。

2. 下一篇预告

本篇讲了哪些通道被激活、每个通道查多少条。但通道内部是怎么跑的——IntentParallelRetriever 怎么对多个 Collection 做并行检索、多个通道的结果怎么合并去重、SearchResultPostProcessor 后置处理器链怎么串联、Reranker 怎么做最终精排——这些细节刻意没展开。

下一篇进入检索阶段的内部实现——多通道并行检索。把 SearchChannel 接口的设计、模板方法模式的并行检索、后置处理器链的串联机制、Reranker 的精排逻辑，一个个拆开来看。

